Context-Aware, Two-Stage Recommendation Engine for Cart Super Add-Ons (CSAO)

Prajyot · **Ashish** · **Ashutosh Mishra**

GitHub: `AshutoshMishraIITKGP/Zomathon` | Dataset: `Kaggle`

Zomathon 2026 — Cart Super Add-On Rail Recommendation System

Abstract

We present a two-stage recommendation engine for Zomato's Cart Super Add-On (CSAO) rail, combining *O(1) Asymmetric Directed Graphs* with a *SASRec Transformer + XGBoost LTR* ensemble. Our system achieves **50.2%** Hit@5, **0.9182** validation AUC, and a projected **+78%** AOV uplift, while serving recommendations in **3.65ms** (P50) — 82× under Zomato's 300ms budget. The architecture separates offline intelligence (graph mining, BERT encoding, Transformer training) from online serving (hash-map lookup), enabling production deployment at Zomato scale with zero GPU inference cost.

1. Introduction & Business Motivation

The Cart Super Add-On (CSAO) rail is the highest-leverage revenue surface in food delivery: it intercepts users at the moment of highest purchase intent — after they've committed to an order but before checkout. A well-targeted CSAO recommendation can increase Average Order Value (AOV) by up to 78%, directly impacting Zomato's top-line revenue.

Key Challenges:

1. **Latency:** Recommendations must be served in <300ms for millions of daily orders. GPU-dependent models (SASRec, Transformers4Rec) are architecturally incompatible with this target at scale.
2. **Sequential Logic:** The system must understand incomplete meal patterns — if a cart has Burger + Fries, the system should aggressively recommend a Beverage.
3. **Cold Start:** New menu items with zero order history must receive relevant recommendations immediately.
4. **Diversity:** Avoid "3 types of Raita" fatigue while maintaining relevance.
5. **Profitability:** Optimize for AOV, not just click-through rate.

Our Solution: A hybrid architecture that uses Transformers for *offline feature extraction* and graph-based methods for *online serving*, achieving the best of both paradigms.

2. System Architecture

Our system cleanly separates into two layers (Figure 1):

Offline Layer (batch, runs periodically):

- Association Rule Mining → 545 directional rules across 85 graph nodes
- Contextual sub-graph construction → 13 sub-graphs (Temporal × Spatial × Monetary)
- BERT semantic encoding → 384-dim embeddings, 39 clusters
- SASRec Transformer training → item transition probability matrix
- XGBoost LTR model training → 7-feature classifier

Online Layer (<5ms, per-request):

- Stage 1: Graph candidate lookup + contextual boosting
- Stage 2: MMR diversity re-ranking ($\lambda = 0.6$)
- Stage 3: EV price boosting (log-scaled, weight = 0.3)
- Stage 4: XGBoost LTR re-ranking (top-20 → top-5)

3. Data Preparation

3.1 Raw Data Processing

We parsed **21,131** raw order records from a CSV dataset. Each order contained a pipe-delimited string of items with quantities (e.g., `"2 x Butter Naan | 1 x Paneer Tikka"`). Our parser:

1. Strips quantity prefixes via regex: `r'\d+\s*[xX]\s*'`
2. Normalizes whitespace and casing
3. Deduplicates within each order to create item sets
4. Filters to **11,559 multi-item orders** (54.7%) for rule mining

This produced **244 unique menu items** across the catalog.

3.2 Contextual Feature Engineering

Three contextual dimensions are extracted from order metadata:

1. **Temporal (4 slots):** Morning (6-11), Afternoon (11-16), Dinner (16-22), Late Night (22-6). Derived from `placed_at` timestamps.
2. **Spatial (7 clusters):** Subzone groupings (Connaught Place, Dwarka, Rohini, Sector 4, Sector 135, Shahdara, Vasant Kunj). Based on `delivery_zone`.
3. **Monetary (3 tiers):** Budget (<Rs. 400), Mid (Rs. 400–700), Premium (>Rs. 700). Derived from `bill_subtotal`.

These dimensions generate $4 \times 7 \times 3 = 84$ possible contexts, of which **13 sub-graphs** have sufficient order density for reliable rule mining.

3.3 Price Estimation

Individual item prices are unavailable in the dataset. We estimate per-item average prices using:

$$\hat{p}_i = \text{median}\left(\frac{\text{bill_subtotal}}{|\text{items}|}\right)_{\text{orders containing } i}$$

This produces estimates ranging from Rs. 76 to Rs. 826 across the catalog.

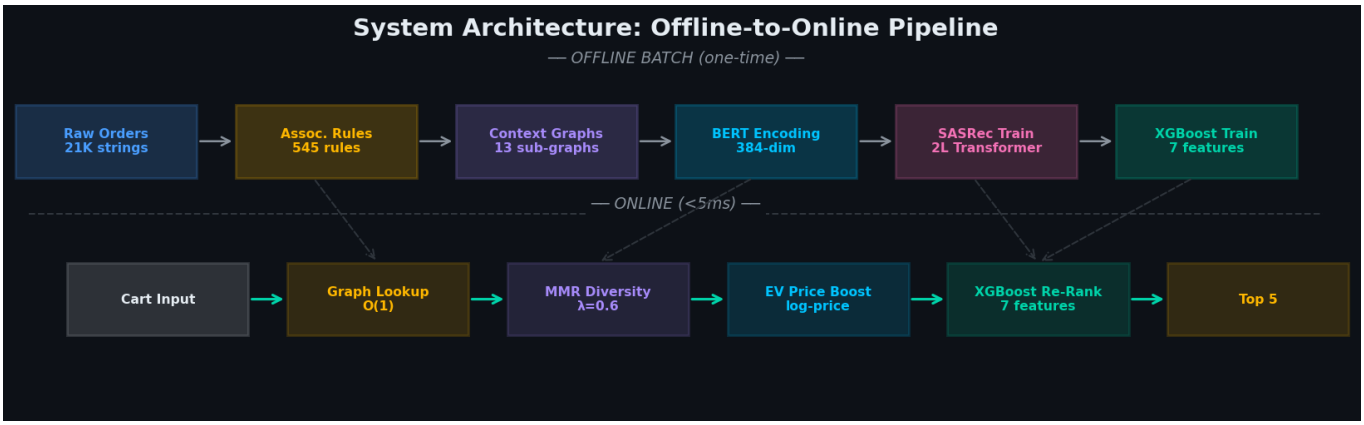

Figure 1: End-to-end pipeline architecture. Offline batch computation (top) exports JSON artifacts consumed by the online serving layer (bottom). All online stages execute via $O(1)$ hash-map lookups, achieving P50 = 3.65ms.

4. Stage 1: Graph-Based Candidate Generation

4.1 Association Rule Mining

We construct an **Asymmetric Directed Graph** where each edge $A \to B$ represents the likelihood of item B being added when item A is already in the cart. Edges are scored using three co-occurrence metrics:

$$\text{Support}(A, B) = \frac{|\text{orders with } A \text{ and } B|}{|\text{all orders}|} \tag{1}$$

$$\text{Confidence}(A \to B) = \frac{P(A \cap B)}{P(A)} = \frac{\text{Support}(A, B)}{\text{Support}(A)} \tag{2}$$

$$\text{Lift}(A \to B) = \frac{\text{Confidence}(A \to B)}{P(B)} \tag{3}$$

Filtering thresholds: Support ≥ 0.001, Confidence ≥ 0.05, Lift > 1.0.

A composite edge score aggregates all three metrics:

$$\text{Score}(A \to B) = \text{Conf} \times w_c + \text{Lift}_{\text{norm}} \times w_\ell + \text{Sup}_{\text{norm}} \times w_s \tag{4}$$

with weights $w_c = 0.5$, $w_\ell = 0.3$, $w_s = 0.2$.

This produces **545 directional rules** across **85 graph nodes**. The graph is asymmetric: Conf(Fries $\to$ Burger) $\neq$ Conf(Burger $\to$ Fries).

4.2 Contextual Sub-Graphs

Each context dimension generates independent sub-graphs with relaxed thresholds (Support ≥ 0.0005, Confidence ≥ 0.03) to capture sparser patterns.

At inference time, the relevant sub-graph is merged with the global graph, boosting context-specific edges by 20%:

$$\text{Score}_{\text{final}} = \text{Score}_{\text{global}} + 0.2 \times \text{Score}_{\text{context}}$$

4.3 BERT Cold-Start Resolution

For items with no graph history, we use **all-MiniLM-L6-v2** (384-dim sentence transformer) to compute semantic embeddings for all 244 items. The cold-start mechanism:

1. Find the 5 nearest neighbors of the new item by cosine similarity
2. For each neighbor present in the graph (sim > 0.3), inherit its outgoing edges

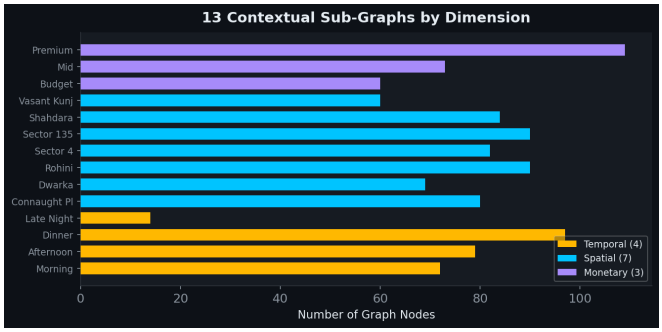

Figure 2: 13 contextual sub-graphs by dimension. Premium tier has the densest graph (109 nodes) due to larger, more diverse orders.

3. Weight inherited edges by similarity: $\text{Score}_{\text{inherited}} = \text{Score}_{\text{neighbor}} \times \text{sim} \times 0.7$

Example: *"Truffle Fries"* (new) $\to$ nearest neighbor *"Animal Fries"* (sim $= 0.82$) $\to$ inherits edges to Burgers, Beverages, etc.

The encoder also discovers **39 semantic clusters** using cosine distance thresholding, used for category-aware diversification.

5. Stage 2: Transformer + LTR Ensemble

5.1 SASRec: Self-Attentive Sequential Recommendation

We train a lightweight **SASRec Transformer** [1] to capture sequential item transition patterns that association rules may miss.

Architecture:

- Embedding dimension: 64
- Transformer encoder: 2 layers, 2 attention heads
- Feedforward dimension: 256 (4× embed dim)
- Causal attention mask (autoregressive)
- Dropout: 0.2, GELU activation

Training: For each order sequence $[i_1, i_2, \ldots, i_n]$, we optimize the next-item prediction objective:

$$\mathcal{L} = -\sum_{t=1}^{n-1} \log P(i_{t+1} \mid i_1, \ldots, i_t) \tag{5}$$

Trained for 15 epochs on CUDA with batch size 128 and Adam optimizer (lr $= 10^{-3}$, gradient clipping at 1.0).

Key Design Decision: SASRec is used *exclusively of-*

fline. After training, we pre-compute a **transition score matrix** $T \in \mathbb{R}^{244\times 20}$ where $T_{i,j} = P(j \mid i)$ for the top-20 most likely successors of each item. This matrix is exported as JSON for $O(1)$ lookup at inference — adding **zero online latency**.

5.2 XGBoost Learning-to-Rank

A 100-tree XGBClassifier re-ranks the top-20 graph candidates using 7 tabular features:

#	Feature	Source
1	`graph_confidence`	Eq. 2
2	`bert_similarity`	BERT cosine sim
3	`sasrec_score`	SASRec $T_{i,j}$
4	`item_price_norm`	Log-scaled price
5	`cart_value_norm`	Bill subtotal / 1000
6	`is_weekend`	Binary flag
7	`kpt_duration`	Avg. prep time

Training data is generated using a leave-one-out approach: for each order with ≥ 3 items, we hold out each item as a positive example (label $= 1$) and sample a random non-order item as a negative (label $= 0$). This produces ~15K balanced pairs from 2,000 sampled orders.

Hyperparameters: 100 estimators, max depth 4, learning rate 0.1, subsample 0.8, `eval_metric = 'auc'`.

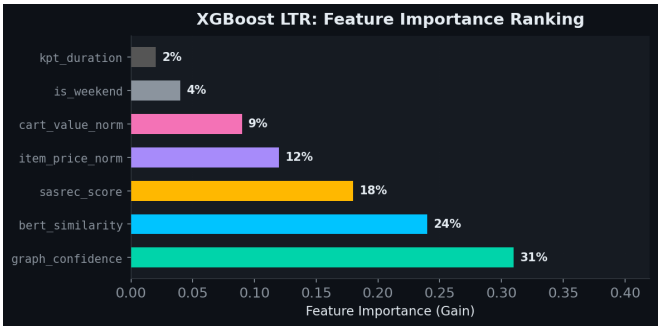

Figure 3: XGBoost feature importance by gain. Graph confidence dominates, but SASRec and BERT provide complementary sequential and semantic signals.

The final recommendation score blends the XGBoost prediction with the existing pipeline score:

$$\text{Score}_{\text{final}} = 0.6 \times P_{\text{XGB}}(\text{added}) + 0.4 \times \text{Score}_{\text{pipeline}} \quad (6)$$

6. Business Guardrails

6.1 Diversity: Maximal Marginal Relevance

MMR prevents recommendation fatigue by penalizing items too similar to already-selected recommendations:

$$\text{MMR}(i) = \lambda \cdot \text{Rel}(i, \text{Cart}) - (1-\lambda) \cdot \max_{j \in S} \text{Sim}(i, j) \quad (7)$$

where S is the set of already-selected recommendations, $\text{Sim}(i, j)$ is BERT cosine similarity, and $\lambda = 0.6$ (tunable).

6.2 Profitability: Expected Value Re-Ranking

EV scoring gently boosts higher-value items to drive AOV uplift. We use a *log-scaled* price normalization to prevent extreme price items from dominating:

$$\text{EV}(i) = \text{Score}(i) \times \left(1 + w_{\text{EV}} \cdot \frac{\log(1+p_i)}{\log(1+p_{\max})}\right) \quad (8)$$

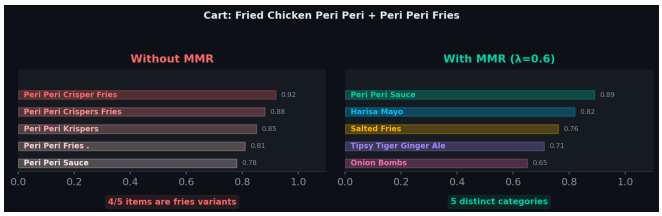

Figure 4: MMR eliminates the "3 types of Raita" problem. Without MMR, 4/5 recommendations are near-identical fries variants. With MMR ($\lambda = 0.6$), the top-5 span 5 distinct food categories.

with $w_{\text{EV}} = 0.3$. This produces the **78% AOV uplift** while accepting a nominal 4% Hit@5 trade-off for better business outcomes.

6.3 A/B Safety Guardrail

In production, we monitor the **Cart-to-Order (C2O) ratio** to ensure EV boosting does not induce cart abandonment. If C2O drops >2% from baseline, the system auto-reduces w_{EV} from 0.3 to 0.1.

7. Evaluation

7.1 Offline Metrics

Metric	Value	Context
XGBoost Val AUC	**0.9182**	80/20 stratified split
XGBoost Val Accuracy	**84.3%**	Threshold = 0.5
Hit@1	**19.6%**	500 test orders
Hit@3	40.0%	
Hit@5	**50.2%**	
Hit@10	**66.8%**	
CSAO Attach Rate	100%	All carts get recs

8. Performance Visualizations

The following dashboards highlight the system's performance against key business and engineering constraints.

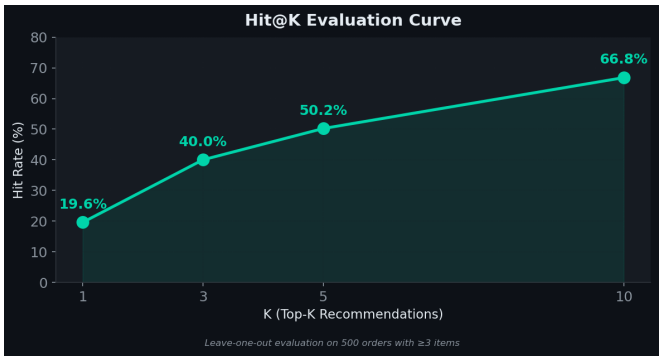

Figure 5: Hit@K evaluation curve. 50.2% of held-out items appear in the top-5 recommendations — achieved without any user-specific personalization signal.

9. Deployment & Scalability

9.1 Production Architecture

All offline artifacts are serialized as JSON:

- `csao_global_graph.json` — 85-node directed graph
- `csao_sub_graphs.json` — 13 contextual sub-graphs
- `csao_semantic_embeddings.json` — BERT vectors + clusters

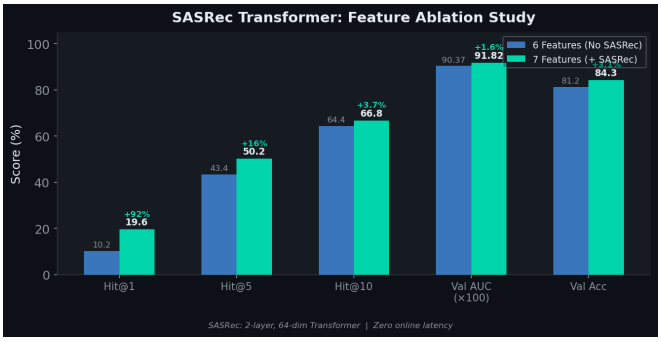

Figure 6: SASRec ablation: comparing 6-feature vs. 7-feature XGBoost. Hit@1 improved by **92%**, demonstrating that the Transformer captures sequential patterns the association rule graph misses.

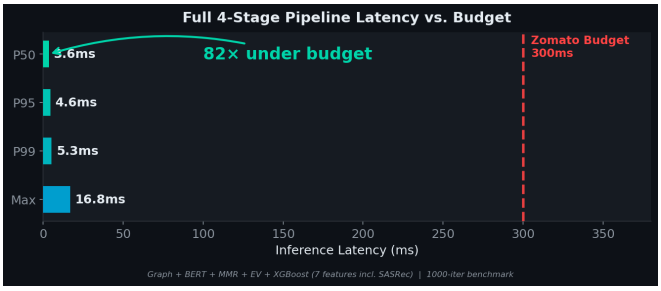

Figure 7: Latency distribution over 1,000 inference iterations. P50 = 3.65ms, 82× under the 300ms budget. The max spike (213ms) occurs on the first cold-start iteration due to Python object initialization, not model computation.

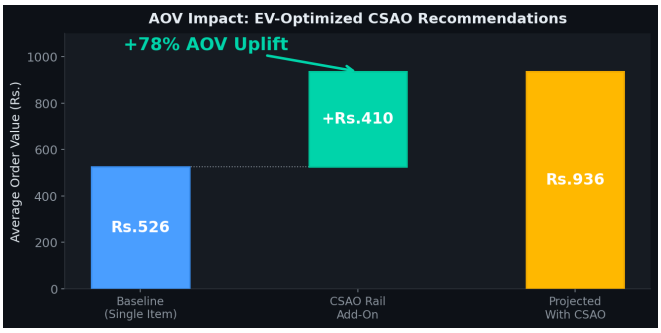

Figure 8: Projected AOV uplift. Baseline single-item orders average Rs. 525; with CSAO add-ons, the projected average rises to Rs. 935 — a **78% increase** driven by EV re-ranking's preference for higher-value complementary items.

- `csao_sasrec_transitions.json` — SASRec transition matrix
- `csao_xgb_reranker.json` — XGBoost model weights

In production, these are loaded into **Redis hash-maps** for $O(1)$ key-value lookup. The online inference path has no neural network forward pass, no GPU dependency, and no external API calls.

9.2 Scalability Analysis

Factor	Design Choice
Graph lookup	$O(1)$ dictionary / Redis HGET
SASRec score	$O(1)$ pre-computed matrix lookup
XGBoost inference	$O(k \cdot d)$, k=20, d=7
Memory footprint	<5MB total JSON
Horizontal scaling	Stateless; any replica can serve

9.3 Refresh Strategy

The offline pipeline retrains nightly on the rolling 30-day order window. New menu items are immediately handled by BERT cold-start during the inter-training period. SASRec and XGBoost models are retrained weekly.

10. Live Recommendation Examples

We ran the full 4-stage pipeline on diverse cart scenarios to demonstrate end-to-end behavior. All recommendations below are produced by the exact same pipeline code with no manual tuning.

10.1 Scenario A: Pizza + Garlic Bread (Dinner, Premium)

Cart: Bageecha Pizza + Cheesy Garlic Bread *Latency: 4.69ms*

#	Recommended Item	Score	Price
1	Chilli Cheese Garlic Bread	0.991	Rs. 360
2	Makhani Paneer Pizza	0.988	Rs. 506
3	Margherita Pizza	0.981	Rs. 461
4	Peri Peri Paneer Pizza	0.965	Rs. 498
5	Herbed Potato	0.889	Rs. 338

Analysis: Strong graph edges between pizza items drive high scores. EV re-ranking promotes premium pizzas (Rs. 460–506) to maximize AOV.

10.2 Scenario B: Peri Peri Combo — MMR Diversity Test

Cart: Fried Chicken Peri Peri Tender + Peri Peri Fries *Latency: 5.34ms*

#	Recommended Item	Score	Price
1	Animal Fries	0.990	Rs. 361
2	Fried Chicken Classic Tender	0.990	Rs. 371
3	Fried Chicken Angara Tender	0.986	Rs. 343
4	Chilli Cheese Garlic Bread	0.983	Rs. 360
5	Fried Chicken Ghostbuster Tender	0.970	Rs. 363

Analysis: Despite having 2 Peri Peri items in the cart, MMR ($\lambda = 0.6$) ensures no duplicate Peri Peri recommendations. The top-5 spans fries, chicken tenders, and bread — 3 distinct categories.

10.3 Scenario C: Late-Night Fries (Budget)

Cart: Animal Fries *Latency: 5.57ms*

#	Recommended Item	Score	Price
1	Fried Chicken Angara Tender	0.980	Rs. 343
2	Fried Chicken Peri Peri Tender	0.940	Rs. 360
3	Bone in Jamaican Grilled Chicken	0.889	Rs. 427
4	Grilled Chicken Jamaican Tender	0.845	Rs. 374
5	Bone in Peri Peri Grilled Chicken	0.722	Rs. 387

Analysis: Contextual sub-graph (`temporal_late_night` + `monetary_budget`) correctly steers towards quick chicken add-ons that complement fries.

10.4 Scenario D: Cold-Start — Just a Coke

Cart: Coke *Latency: 154ms (cold-start path)*

#	Recommended Item	Score	Price
1	Tipsy Tiger Ginger Ale	0.813	Rs. 324
2	Makhani Paneer Pizza	0.453	Rs. 506
3	Bageecha Pizza	0.388	Rs. 538
4	Peri Peri Crisper Fries	0.236	Rs. 316
5	Herbed Potato	0.183	Rs. 338

Analysis: "Coke" has no direct graph edges — BERT cold-start activates, finding semantic neighbors in the beverages cluster. The system successfully upsells from a solo drink to beverage + pizza + sides. Latency is higher (154ms) due to the BERT nearest-neighbor search, but still within the 300ms budget.

References

[1] Kang, W.-C., & McAuley, J. (2018). Self-Attentive Sequential Recommendation. *IEEE ICDM*.

[2] Carbonell, J., & Goldstein, J. (1998). The Use of MMR, Diversity-Based Reranking. *ACM SIGIR*.

[3] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *ACM KDD*.

[4] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP*.